

Aureus

A Multi-Agent AI Accounting Assistant

University of Wisconsin–Madison • CS 620

Capstone Project Report • Industry Partner: **Denari**

Farhan Adnan • Fairoz Khan • Anna Wang • Sanjana Reddy • Travis Johnson • Tara Awate

At a glance. Aureus is a coordinated three-agent AI system that gives small-business owners and their accountants rapid, on-demand answers to tax and financial questions — without booking a formal consultation. A **Main Orchestration Agent** routes each request to a **Customer Knowledge Agent** for the client’s internal financial data and a **Web Search Agent** for authoritative external context such as IRS rules and accounting standards, then assembles a single grounded answer. A human-in-the-loop review step gates anything that writes to the client’s books.

3

coordinated agents

HITL

review before any write

Live

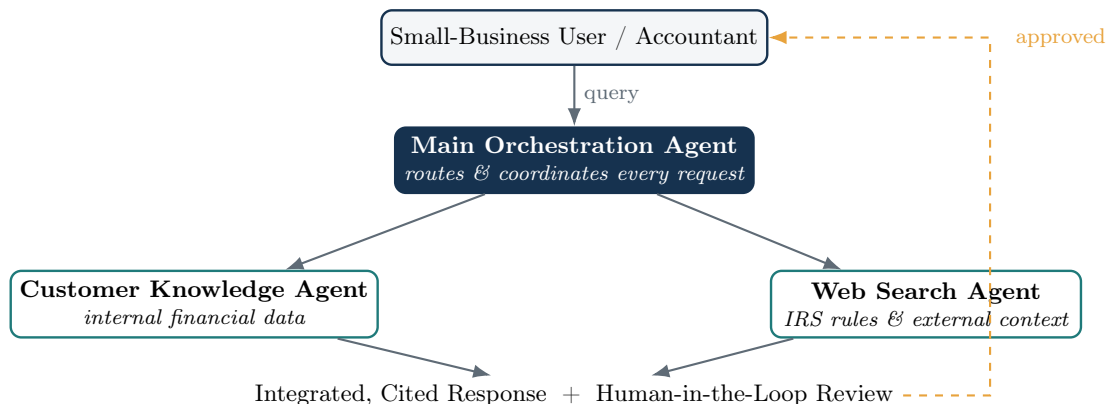
streaming responses

Problem & Approach

Small-business owners routinely face tax and bookkeeping questions that are too involved for a quick web search but too small to justify scheduling an accountant. General-purpose, single-agent chat assistants struggle here: a single model asked to simultaneously recall a client’s ledger, retrieve current IRS guidance, and reason about the two at once tends to blur or hallucinate across those distinct sub-tasks.

Aureus addresses this by **decomposing the problem across specialists**. Rather than one model doing everything, an orchestrator delegates each part of a query to the agent best suited for it — internal financial reasoning to one, external regulatory lookup to another — and composes their outputs into a coherent, cited answer. The design goal was not just to answer questions, but to answer them in a way an accounting firm could actually trust: grounded in real data, backed by authoritative sources, and never silently altering a client’s records.

System Architecture



The orchestrator acts as a project manager: it interprets the user’s intent, decides which specialist(s) to invoke, and merges their results. Splitting responsibilities this way keeps each agent’s context focused — the Customer Knowledge Agent reasons only over the client’s books, the Web Search Agent only over trusted external sources — which improves both answer quality and traceability.

The Three Agents

- **Main Orchestration Agent.** Receives the user prompt, classifies the request, routes work to the appropriate specialist(s), and assembles the final response. It also owns the transaction-confirmation flow, previewing any proposed change to a client's books before it is committed.
- **Customer Knowledge Agent.** Reads and reasons over the client's internal financial records — account balances, transaction history, and company-specific data — to answer questions grounded in that specific business's numbers.
- **Web Search Agent.** Retrieves authoritative external context (IRS regulations, tax deadlines, accounting definitions) so answers reflect current, real-world guidance rather than the model's stale internal knowledge.

Key Capabilities

Beyond the core question-answering loop, Aureus grew into a more complete product over the semester:

- **Streaming responses.** Output renders token-by-token as an agent generates it, so users see progress immediately instead of waiting for a full answer.
- **Company selection.** A user can scope a conversation to one specific business, so every answer is grounded in the right client's books.
- **Review-before-write workflow.** Any change destined for the database is surfaced for human review and approval first — the system never silently mutates a client's records.
- **Financial snapshots.** On-demand balance-sheet views summarize current assets, liabilities, and equity, plus the ability to update a client's books.
- **Accounts & access.** Login/signup and per-account data scoping support a realistic, multi-client product experience.

Engineering Process

The team began the semester on **Scrum** — Farhan as Scrum Master, Sanjana as Product Owner — running stand-ups, sprint transitions, retrospectives, and recorded demos. As the system matured, the process evolved into **Scrumban**: sprint-based planning and stand-ups were retained, but members worked independently on feature branches and pulled new tickets from the Jira board as soon as they finished current work. This hybrid let the team build and integrate the three-agent architecture and UI in time for the mid-semester science fair, then iterate continuously afterward.

Practice	How the team used it
Stand-ups	Twice weekly (≈ 15 min): completed work, live feature demos, next steps.
Planning	Sprint-based, closed and opened on a recurring cadence; async updates via Slack when needed.
Tracking	Jira as the single board for active features and sprint tasks; tickets pulled continuously.
Version control	Individual GitHub branches, marked for review, merged into <code>main</code> once approved.
Mentorship	Recurring mentor meetings with the Denari industry partner.

Challenges & How We Solved Them

Untracked, overlapping work. The freedom of Scrumban occasionally led to features being started without a corresponding Jira ticket, and to two people unknowingly building the same thing on separate branches. *Fix:* we assigned explicit feature ownership before each sprint and communicated branch status more actively in Slack.

Integration ambiguity. With everyone on individual branches, it wasn't always clear which commits were ready to merge, which slowed early agent integration. *Fix:* we adopted a lightweight review-and-mark convention before merging approved work into `main`.

No roadmap after the core shipped. Because much of the system already worked by the science fair — and our mentor gave broad latitude on direction — the team briefly lacked a clear “what next.” *Fix:* we returned to user-interview notes and let real requests drive the backlog, which directly motivated company-specific conversations, book updates, and balance-sheet snapshots.

Lessons Learned

If we continued Aureus, three process changes would compound the speed Scrumban gave us with better alignment:

- **A dedicated real-time channel** to cut the notification lag we sometimes hit on Slack.
- **Restored per-sprint retrospectives**, so friction is surfaced and fixed before it persists across sprints.
- **More rigorous Jira + written agendas/notes**, so decisions, ownership, and future ideas live in one durable place.

Conclusion

Building Aureus taught the team as much about shipping software collaboratively as it did about multi-agent AI. Moving from Scrum to Scrumban let us deliver a genuinely complex, three-agent system on a tight timeline, while the coordination hurdles along the way underscored how much consistent communication, disciplined version control, and clear ownership matter on a real product. The result is a functional, trustworthy accounting assistant for small-business owners — and hands-on experience across agile delivery, multi-agent AI system design, and team-based engineering.